GUIDE DE SOUTENANCE

Comprendre, présenter et défendre son projet sans être développeuse

Conception et réalisation du site web de Wiky Holding

BTS Informatique Développeur d'Application — ES2I Plateau — 2025-2026

Candidate : KOUAME AFFOUE CYNTICHE ACHLEY DELMAS

Ce que contient ce guide

- Les **pitchs** prêts à dire (30 secondes, 2 minutes, 5 minutes) et un **plan de soutenance** minuté.
- Toutes les **bases de la programmation expliquées simplement**, avec des images de la vie de tous les jours.
- **MERISE** expliquée pas à pas avec ton projet (dictionnaire, règles de gestion, SAT, MCD, MLD).
- **Plus de 50 questions probables du jury** avec des réponses courtes que tu peux dire presque telles quelles.
- Un **scénario de démonstration**, les **pièges à éviter**, et une **fiche mémo** à relire la veille.

Comment l'utiliser

1. Lis d'abord la **partie 1** (ton projet en clair) et la **partie 2** (les mots à connaître).

2. Apprends les **pitchs** de la partie 3 presque par cœur : c'est ce qui rassure le jury dans les 30 premières secondes.

3. Lis la **partie 5** (questions/réponses) à voix haute, avec tes propres mots. Un jury préfère une réponse simple et honnête à une réponse compliquée récitée.

4. La veille : relis uniquement la **fiche mémo** (dernière page) et fais la démo une fois en entier.

Sommaire

1. **Partie 1** — Ton projet expliqué à quelqu'un qui ne connaît rien
2. **Partie 2** — Les bases de la programmation, sans jargon
3. **Partie 3** — Pitchs et plan de la soutenance
4. **Partie 4** — MERISE expliquée avec ton projet
5. **Partie 5** — Les questions du jury et les réponses
6. **Partie 6** — La démonstration, pas à pas
7. **Partie 7** — Conseils du jour J et fiche mémo

Partie 1 — Ton projet en clair

1.1 L'histoire en une minute

Tu as fait ton stage à **Digital Agency Wiky** (aussi appelée **Wiky Holding**), une agence de communication à Abidjan, quartier Cocody Riviera Faya. L'agence fait du community management, du graphisme, de la vidéo et du podcast pour ses clients.

L'agence avait besoin d'un **site web professionnel** pour se présenter et pour **recevoir des demandes de clients** en ligne. Ta mission : **le concevoir et le réaliser**. Tu as d'abord essayé avec WordPress, mais ce premier site n'est plus disponible. Tu l'as donc **reconstruit entièrement toi-même en PHP** avec une base de données MySQL.

1.2 Ce que fait le site, avec une image

Imagine l'agence comme **un magasin**. Le site web est la **vitrine** : n'importe qui peut regarder, de jour comme de nuit. À l'intérieur, il y a deux guichets :

- **Le guichet « Contact »** : le visiteur écrit un message.
- **Le guichet « Devis »** : le visiteur simule un prix, puis envoie une demande de devis.

Derrière la vitrine, il y a **un bureau privé** (le **Back-Office**, ou espace administrateur) : seul le gestionnaire de l'agence y entre avec un mot de passe. Il y voit les messages et les demandes de devis reçus, et peut les traiter.

Toutes ces informations sont rangées dans **la base de données**, un grand classeur numérique organisé en 5 tiroirs (les 5 tables).

> **Si tu ne retiens que trois phrases**
>
> - « J'ai conçu **un site web avec un espace administrateur** pour que l'agence reçoive et traite les demandes de ses clients. »
> - « J'ai **modélisé les données avec MERISE** avant de programmer, pour que la base soit propre. »
> - « J'ai **sécurisé** le site contre les attaques web les plus courantes. »

1.3 Les chiffres à connaître par cœur

Élément	Valeur	Où le retrouver
Pages publiques	**7** pages (Accueil, À propos, Services, Réalisations, Studio podcast, Contact, Devis) + une page d'erreur 404	Partie II, chap. I
Tables de la base	**5** : services, prospects, admins, messages, devis_requests	Chap. IV, §4.4
Propriétés (dictionnaire)	**26** propriétés, sans doublon	Chap. II, §2.2
Règles de gestion	**6** (RG1 à RG6)	Chap. II, §2.3
Associations du MCD	**6** : Émettre, Formuler, Concerner, Porter sur, Modérer, Instruire	Chap. II, §2.5
Langage / SGBD	**PHP 8.2 à 8.4** et **MySQL** (moteur InnoDB)	Chap. IV

Élément	Valeur	Où le retrouver
Architecture	**MVC** (Modèle - Vue - Contrôleur)	Chap. III
Sécurité	Requêtes préparées PDO, jeton CSRF, échappement XSS, mot de passe haché BCRYPT, .htaccess	Chap. VII
Tests	**14** scénarios, tous « Conforme »	Chap. VIII
Outils / hébergement	WampServer 3.4.0 en local, InfinityFree en ligne	Chap. IX
Coût	**0 FCFA** (tout est libre et gratuit)	Chap. IX, §9.3

Partie 2 — Les bases de la programmation, sans jargon

Tu n'as pas besoin de savoir coder pour défendre ce projet. Tu as besoin de **comprendre ce que chaque mot veut dire** et de **savoir l'expliquer avec une image simple**. Voici le vocabulaire, du plus général au plus technique.

2.1 Comment un site web fonctionne

Mot	Explication simple	Dans ton projet
Site web	Un ensemble de pages qu'on consulte avec un navigateur (Chrome, Firefox…).	Le site de Wiky Holding.
Client / Serveur	Le **client** est celui qui demande (le navigateur du visiteur). Le **serveur** est celui qui répond (l'ordinateur qui héberge le site). C'est comme un client et un serveur au restaurant.	Ton PC avec WampServer joue le serveur en local ; InfinityFree en ligne.
Requête HTTP	Le « message » envoyé par le navigateur au serveur : « donne-moi la page contact ».	Exemple : index.php?page=contact
HTML	Le **squelette** de la page : titres, paragraphes, boutons.	Les vues du site.
CSS	Le **maquillage** : couleurs, polices, espaces, mise en page.	Bleu marine + orange corail, thèmes clair/sombre.
JavaScript	Ce qui rend la page **vivante** sans la recharger : filtres, calculs, menus.	Simulateur de devis, recherche Ctrl+K, filtre des réalisations.
PHP	Le langage qui tourne **sur le serveur** : il reçoit la demande, lit la base de données, prépare la page et l'envoie.	Tout le « cerveau » du site.
Base de données	Un **classeur numérique** qui range les informations en tableaux.	Prospects, messages, devis, services, admins.
MySQL	Le logiciel qui gère la base de données (un SGBD : Système de Gestion de Base de Données).	Base nommée wiky_db.
InnoDB	Le « moteur » de MySQL qui garantit que les données restent **fiables** et que les liens entre tables sont respectés.	Utilisé pour les 5 tables.
phpMyAdmin	Une interface web pour **voir et modifier** la base sans écrire de code.	Tes captures d'écran des figures 3 à 7.
WampServer	Un logiciel qui transforme ton PC Windows en **petit serveur** (Apache + MySQL + PHP).	Pour tester le site en local.
Hébergement	Louer une place sur un serveur toujours allumé pour que tout le monde voie le site.	InfinityFree (gratuit).
HTTPS / SSL	Le **cadenas** : les échanges sont chiffrés, personne ne peut les lire en route.	Certificat SSL gratuit d'InfinityFree.
Apache et .htaccess	Apache est le logiciel serveur. Le fichier .htaccess est son **règlement intérieur** : il bloque certains fichiers et ajoute des protections.	Chap. VII, §7.4.

Schéma à retenir (une demande de page) : le visiteur clique → le navigateur envoie une requête → le serveur exécute le PHP → PHP va chercher les données dans MySQL → PHP fabrique la page HTML → le navigateur l'affiche.

2.2 Programmer proprement : les notions du rapport

Mot	Explication simple	Dans ton projet
POO (programmation orientée objet)	On range le code en **classes** : chaque classe a un métier précis, comme chaque employé d'une agence.	Contrôleurs, modèles, classe Database.
MVC	Séparer le code en 3 rôles. Voir l'image du restaurant ci-dessous.	Dossiers Controllers, Models, Views.
Routeur / contrôleur frontal	Le **réceptionniste** : il lit l'adresse demandée (?page=contact) et envoie le visiteur vers le bon contrôleur.	Le fichier index.php.
Singleton	On ne crée **qu'une seule connexion** à la base, et tout le monde la partage (au lieu d'en ouvrir cent).	Classe Database.php.
PDO	Le « traducteur » standard qui permet à PHP de parler à MySQL de façon sécurisée.	Toutes les requêtes passent par PDO.
Requête préparée	On envoie d'abord le **modèle** de la question SQL, puis les **valeurs** séparément. Une valeur ne peut jamais être prise pour une commande.	Protection contre l'injection SQL.
CRUD	Les 4 actions de base sur des données : **C**réer, **R**ead (lire), **U**pdate (modifier), **D**elete (supprimer).	Gestion des messages et devis dans le Back-Office.
Session	La « mémoire » du serveur pour se souvenir qu'un visiteur est connecté.	Connexion de l'administrateur.
Responsive design	Le site **s'adapte** à la taille de l'écran (téléphone, tablette, PC).	Figure 28 (version mobile).
localStorage	Une petite mémoire dans le navigateur du visiteur.	Il retient le thème choisi (clair, sombre, bleu marine).
Framework	Une boîte à outils toute prête (Laravel, Symfony…). Tu n'en as **pas utilisé** : code écrit à la main, plus léger.	Choix assumé du « code propriétaire ».

L'image du restaurant pour expliquer le MVC

Un client commande au restaurant. **Le serveur (Contrôleur)** prend la commande et décide à qui la transmettre. **Le cuisinier (Modèle)** va chercher les ingrédients dans la réserve (la base de données) et prépare le plat. **L'assiette dressée (Vue)** est ce que le client voit.

Dans ton site : « Quand on ouvre la page Contact, le **contrôleur** reçoit la demande, le **modèle** enregistre le message dans MySQL, et la **vue** affiche la page au visiteur. Chacun fait **un seul travail**, donc c'est plus facile à corriger et à faire évoluer. »

2.3 La sécurité, expliquée avec des images

Le chapitre VII est celui qui impressionne le jury. Voici chaque attaque en une phrase, et ta parade.

Attaque	Image simple	Ta parade
Injection SQL	Un voleur écrit une commande pirate dans un formulaire (par exemple dans le champ « nom ») pour que la base de données lui obéisse.	**Requêtes préparées PDO**, avec émulation désactivée : les valeurs sont envoyées séparément et ne sont jamais exécutées comme des ordres.
CSRF	Quelqu'un te fait envoyer un formulaire **à ton insu** pendant que tu es connectée (comme falsifier ta signature).	**Jeton secret** aléatoire de 32 octets (random_bytes) mis dans chaque formulaire et vérifié par hash_equals. Sans le bon jeton, la requête est rejetée.
XSS	Un pirate glisse un petit programme dans un texte (un message de contact) qui s'exécute chez l'administrateur quand il le lit.	Fonction d'**échappement e()** (htmlspecialchars) : tout texte est affiché comme du texte, jamais comme du code.
Mot de passe volé	Si la base fuit et que les mots de passe sont lisibles, tout est perdu.	**BCRYPT** : le mot de passe est transformé en empreinte irréversible. Même l'administrateur ne peut pas le relire.
Fixation de session	Un pirate garde un ancien « badge » de session.	**session_regenerate_id(true)** : nouveau badge à chaque connexion réussie.
Fuite d'informations	Un message d'erreur qui dit « ce nom n'existe pas » aide le pirate.	Message **unique** : « Identifiants incorrects ». Aucun indice dans le formulaire.
Fichiers sensibles	Quelqu'un tape l'adresse de database.sql dans le navigateur.	**.htaccess** bloque l'accès direct, désactive le listing des dossiers et ajoute des en-têtes de sécurité.

Partie 3 — Pitchs et plan de la soutenance

Un **pitch**, c'est présenter ton projet de façon courte et claire. Apprends les trois versions : elles servent à l'accueil du jury, à la question « présentez-nous votre projet » et à la conclusion.

3.1 Pitch de 30 secondes (la version ascenseur)

À dire

« Bonjour, je m'appelle Kouame Affoue Cyntiche Achley Delmas, étudiante en BTS Informatique Développeur d'Application à l'ES2I Plateau. Pendant mon stage à **Digital Agency Wiky**, à Abidjan, j'ai **conçu et réalisé le site web de l'agence**. Il présente ses services et permet aux clients d'**envoyer un message ou de demander un devis en ligne**. Un **espace administrateur sécurisé** permet ensuite de traiter ces demandes. Je l'ai construit en **PHP et MySQL**, avec la **méthode MERISE** et l'**architecture MVC**. »

3.2 Pitch de 2 minutes

À dire (adapte avec tes mots)

Le contexte. « Aujourd'hui, une entreprise a besoin d'être visible sur Internet. Digital Agency Wiky est une agence de communication à 360 degrés : réseaux sociaux, identité visuelle, vidéo, podcast. Elle avait besoin d'un site moderne pour se présenter et recevoir des demandes de clients. »

Le problème. « La question qui a guidé mon travail était : comment concevoir un site **professionnel, rapide et sécurisé**, reposant sur une **modélisation rigoureuse des données** ? »

Ma solution. « J'ai d'abord **modélisé les données avec MERISE** : dictionnaire de 26 propriétés, 6 règles de gestion, SAT, MCD à 5 entités, puis MLD en 3ème forme normale. Ensuite, j'ai programmé le site en **PHP orienté objet selon le modèle MVC**, avec une base **MySQL** accédée via **PDO**. Le site compte 7 pages publiques, un simulateur de devis en direct et un Back-Office pour gérer messages et devis. »

La sécurité. « J'ai protégé le site contre les injections SQL, les attaques CSRF et XSS, et les mots de passe sont chiffrés en BCRYPT. »

Le résultat. « Le site fonctionne en local sous WampServer et peut être déployé sur InfinityFree. Il a été testé sur 14 scénarios, tous conformes, et il a coûté **0 FCFA**. Je vous propose maintenant de vous le présenter en détail. »

3.3 Pitch de 5 minutes : la structure

Pour une version longue, suis toujours le même fil : **Pourquoi → Quoi → Comment → Résultat → Après**.

Étape	Durée	Ce que tu dis
Pourquoi	1 min	L'agence, son besoin, la problématique.
Quoi	1 min	Les fonctionnalités : 7 pages, contact, devis, Back-Office, recherche, thèmes.

Étape	Durée	Ce que tu dis
Comment	2 min	MERISE puis MVC, MySQL/PDO. Une image par idée (classeur, restaurant).
Résultat	45 s	Sécurité, tests, déploiement, coût 0 FCFA.
Après	15 s	Perspectives : notifications email/SMS, factures PDF, catalogue dynamique, version bilingue.

3.4 Plan conseillé pour la soutenance (environ 15 à 20 minutes)

Demande la durée exacte à ton école et adapte les minutes. L'important est l'ordre.

#	Partie	Durée	Idée-force à faire passer
1	Introduction : qui je suis, thème, plan	1 min	Je sais où je vais.
2	L'entreprise d'accueil	2 min	Agence de communication, 4 pôles, Cocody Riviera Faya.
3	Le projet : contexte, problématique, objectifs	2 min	Un besoin réel, une vraie mission.
4	La modélisation MERISE	4 min	Les données ont été pensées **avant** de coder.
5	Architecture MVC et base MySQL	2 min	Code organisé, séparé, maintenable.
6	Démonstration du site	4 min	Ça marche vraiment (voir partie 6).
7	Sécurité et tests	2 min	Le site est protégé et vérifié.
8	Conclusion et perspectives	1 min	Bilan, ce que j'ai appris, suite possible.

Règles d'or de l'oral

- **Une idée par diapositive**, peu de texte : tu parles, les slides ne se lisent pas.
- Utilise **toujours une image simple** avant le terme technique (classeur, restaurant, badge).
- Dis « **j'ai** » et non « on a » : c'est ton projet.
- Respire, ralentis. Un silence de 2 secondes paraît naturel, pas gênant.
- Regarde le jury, pas l'écran. Garde ton rapport ouvert sous la main.

Partie 4 — MERISE expliquée avec ton projet

4.1 À quoi ça sert ?

MERISE est une méthode française pour **concevoir une base de données avant de programmer**. Image : avant de construire une maison, l'architecte dessine des **plans**. MERISE, ce sont les plans de ta base de données. Si les plans sont bons, la construction est solide.

MERISE avance **niveau par niveau**, du plus abstrait au plus concret :

Niveau	Nom	Question à laquelle il répond	Chez toi
Conceptuel	MCD	Quelles **informations** et quels **liens** existent dans le métier ?	Figure 2 : 5 entités, 6 associations
Logique	MLD-R	Comment les ranger en **tables** (clés primaires et étrangères) ?	Section 2.6
Physique	MPD	Comment c'est **réellement** créé dans MySQL ?	Chap. IV, tables et phpMyAdmin

4.2 Les étapes, une par une

Étape 1 : le domaine d'étude

C'est **la phrase qui dit de quoi on parle**. Le tien : « L'informatisation de la gestion des clients de l'entreprise Wiky Holding ». Tout ce qui est dans ta modélisation doit servir ce domaine, rien de plus.

Étape 2 : le dictionnaire des données

C'est **la liste de toutes les informations** à conserver : le nom d'un prospect, l'email, le budget d'un devis… Chaque information a un **code** (nomProspect), une **signification**, un **type** (A = lettres, AN = lettres et chiffres, N = nombres, Date), une **longueur** et une **nature**.

- **EL** = élémentaire : une donnée qu'on ne peut pas couper en morceaux (on sépare le nom et le téléphone).
- **SIG** = signalétique : décrit quelqu'un ou quelque chose de façon stable (nom, email).
- **SIT** = situation : dépend du moment (statut d'un message, budget).
- **M** = mouvement : une action datée (date d'envoi d'un message).

Règles à citer : aucune donnée **calculable** (pas de total, pas de TVA), aucune propriété en **double**, chaque identifiant a un préfixe selon sa famille (**mat** = personne du personnel, **num** = dossier ou entité externe, **code** = catalogue, **id**, **immat**). C'est pour cela qu'on a matAdmin, numProspect, numMessage, numDevis, codeService.

Étape 3 : les règles de gestion et les cardinalités

Ce sont **les phrases du métier** qui disent comment les choses sont liées. Chaque règle s'écrit en **aller / retour**. Exemple RG1 :

- **Aller :** un prospect émet **zéro ou plusieurs** messages → (0,n) côté PROSPECT.
- **Retour :** un message est émis par **un et un seul** prospect → (1,1) côté MESSAGE.

Lire une cardinalité en 10 secondes

Le **premier chiffre** est le **minimum** (0 = facultatif, 1 = obligatoire). Le **second** est le **maximum** (1 = au plus un, n = plusieurs).

- **(0,n)** : peut en avoir aucun, peut en avoir plusieurs.
- **(1,1)** : en a obligatoirement un, et un seul.
- **(0,1)** : en a au plus un, mais ce n'est pas obligatoire.

Règle	Lien	Cardinalités	Sens métier
RG1	PROSPECT – MESSAGE (Émettre)	(0,n) – (1,1)	Un message vient toujours d'un prospect.
RG2	PROSPECT – DEMANDE_DEVIS (Formuler)	(0,n) – (1,1)	Un devis est demandé par un prospect.
RG3	SERVICE – MESSAGE (Concerner)	(0,n) – (0,1)	Un message peut ne viser aucun service précis.
RG4	SERVICE – DEMANDE_DEVIS (Porter sur)	(0,n) – (1,1)	Un devis porte toujours sur un service.
RG5	ADMINISTRATEUR – MESSAGE (Modérer)	(0,n) – (0,1)	Un message n'est pas forcément encore modéré.
RG6	ADMINISTRATEUR – DEMANDE_DEVIS (Instruire)	(0,n) – (0,1)	Un devis n'est pas forcément encore instruit.

Étape 4 : le GDF et la SAT

Le **GDF (graphe des dépendances fonctionnelles)** répond à une seule question : « **si je connais A, est-ce que je connais B ?** ». Exemple : si je connais le **numéro d'un message**, je connais son objet, son texte, son statut, sa date, **et aussi** le prospect qui l'a écrit, le service visé et l'administrateur qui l'a traité.

La **SAT (Structure d'Accès Théorique)** est le GDF **nettoyé** : on supprime les **raccourcis inutiles** (relations transitives). Image : si on peut aller de A à B, de B à C, et qu'il existe aussi une flèche directe de A à C, cette dernière est un raccourci inutile : on la retire.

Ce que tu dis sur ta SAT (figure 1)

« Sur ma SAT, chaque identifiant est **souligné**, avec ses propriétés dessous. Les **flèches** vont de numMessage et de numDevis vers numProspect, codeService et matAdmin. Il n'y a **aucune flèche entre prospect, service et administrateur**, donc **aucun raccourci à supprimer** : mon GDF est égal à ma SAT. »

Étape 5 : le MCD

Le **MCD (modèle conceptuel de données)** dessine les **entités** (PROSPECT, SERVICE, ADMINISTRATEUR, MESSAGE, DEMANDE_DEVIS) avec leurs propriétés, reliées par les **associations** et leurs cardinalités. Chaque entité a un **identifiant unique** (souligné).

Étape 6 : le MLD-R (passage au logique)

On transforme le MCD en **tables**. Règle principale : quand le lien est « un à plusieurs », **l'identifiant du côté (0,n) est copié comme clé étrangère** dans la table du côté (1,1) ou (0,1). Chez toi, **numProspect, codeService et matAdmin** sont copiés dans **MESSAGE** et **DEMANDE_DEVIS** (notés **#**).

Mot	Explication simple
Clé primaire (PK)	L'**identifiant unique** d'une ligne, comme le numéro de ta carte d'identité. Deux lignes ne peuvent jamais avoir la même.
Clé étrangère (FK, notée #)	Un champ qui **pointe vers la clé primaire d'une autre table**. C'est comme écrire sur un message : « envoyé par le prospect n° 12 ».
Intégrité référentielle	La base **refuse** un lien vers quelque chose qui n'existe pas.

Étape 7 : les formes normales (1FN, 2FN, 3FN)

- **1FN :** une seule valeur par case (pas de liste dans un champ) et une clé primaire.
- **2FN :** chaque information dépend de **toute** la clé, pas d'un morceau.
- **3FN :** chaque information dépend **directement** de la clé, pas d'une autre information. C'est pour cela qu'on sépare le prospect, le service et l'admin dans leurs propres tables.

> **Phrase simple pour la 3FN**
>
> « La 3FN, c'est **une information, un seul endroit**. Le nom d'un prospect est écrit une seule fois dans la table prospects, pas recopié dans chaque message. Si je le corrige, il est corrigé partout. »

Étape 8 : du logique au physique (MySQL)

Les noms MERISE (numProspect) deviennent des colonnes MySQL (num_prospect) avec un type (INT AUTO_INCREMENT, VARCHAR(120)…). **AUTO_INCREMENT** signifie que MySQL numérote **automatiquement** 1, 2, 3… Les clés étrangères sont déclarées avec **ON DELETE SET NULL ON UPDATE CASCADE** : si un prospect est supprimé, ses messages **restent** (le lien devient vide) ; si un numéro change, les liens se mettent à jour.

Partie 5 — Les questions du jury et les réponses

Les réponses ci-dessous sont **courtes et simples** volontairement. Dis-les avec **tes propres mots** : si tu comprends l'idée, tu n'as pas besoin de réciter. Un jury pose rarement plus de 10 à 15 questions : lis tout, mais insiste sur les thèmes **A, B et D**.

> **Si tu ne connais pas la réponse**
>
> - Ne te tais pas et n'invente pas. Dis : « **Je n'ai pas ce détail en tête, mais voici comment je l'ai compris…** » puis explique l'idée générale.
> - Ou : « **Je peux le vérifier dans mon rapport**, page… » et ouvre-le. C'est normal.
> - Reviens toujours à ce que tu maîtrises : le besoin de l'agence, MERISE, le fonctionnement général, la sécurité.

A. Contexte et projet

Q1. Présentez-vous et présentez votre projet en quelques mots.

Utilise le **pitch de 30 secondes** (partie 3). Termine par : « Je vous propose de détailler la modélisation puis de vous montrer le site. »

Q2. Pourquoi avoir choisi ce thème ?

Parce que toute entreprise a aujourd'hui besoin d'une **présence efficace sur Internet**, et que ce projet me permettait d'**appliquer ma formation** de développeuse sur un cas réel, utile à l'agence qui m'a accueillie.

Q3. Quelle est votre problématique ?

« Comment concevoir et développer un site web **professionnel, rapide et sécurisé** pour Wiky Holding, reposant sur une **modélisation rigoureuse des données** et un code PHP compatible avec les serveurs modernes ? »

Q4. Quels étaient vos objectifs ?

Général : concevoir, modéliser, réaliser et mettre en ligne le site pour accroître la visibilité de l'agence et **digitaliser les demandes de prestations**.

Spécifiques : modéliser avec MERISE, structurer en MVC, stocker dans MySQL/InnoDB, sécuriser (CSRF, XSS, injection SQL), créer un simulateur de devis, adapter à tous les écrans, tester et documenter.

Q5. Qui sont les utilisateurs du site ?

Les visiteurs / prospects : ils consultent les pages, envoient un message ou une demande de devis.

L'administrateur (le gestionnaire de l'agence) : il se connecte au Back-Office pour suivre et traiter les demandes.

Q6. Pourquoi ne pas avoir gardé le site WordPress ?

Le premier site, fait avec WordPress, **n'est plus disponible**. J'ai donc décidé de le reconstruire en **code PHP écrit pour ce projet**. Avantages : contrôle total, site léger, sécurité maîtrisée, et c'est ce qui permettait de montrer les compétences du BTS (modélisation, POO, MVC). Inconvénient : cela demande plus de temps qu'un site avec des modules prêts à l'emploi.

Astuce : Reconnaître un inconvénient montre de la maturité.

Q7. Quelle a été votre méthode de travail ?

Par étapes : **recueil des besoins** avec mon maître de stage, **modélisation MERISE**, **conception MVC**, **développement**, **tests**, puis **préparation du déploiement**.

Q8. Avez-vous réalisé tout le projet seule ? Quelle a été votre part ?

Réponds **avec franchise et précision**. Décris ce que **toi** tu as fait et comprends : recueil des besoins, modélisation MERISE, choix techniques, tests, déploiement, rédaction du rapport. Si tu as utilisé des ressources (cours, documentation, outils, aide d'un encadrant), dis-le simplement : « Je me suis appuyée sur… ».

Un jury pardonne l'aide reçue, jamais un mensonge. Ce guide existe pour que tu **comprennes réellement** ton projet.

B. Modélisation MERISE et base de données

Q9. Qu'est-ce que MERISE et pourquoi l'avoir utilisée ?

C'est une **méthode de conception** qui permet de dessiner les données **avant** de programmer, comme les plans d'une maison. Elle évite les doublons, les oublis et les erreurs de structure dans la base.

Q10. Quelle différence entre MCD, MLD et MPD ?

MCD (conceptuel) : le métier, sans technique : entités, associations, cardinalités.

MLD-R (logique) : les **tables** avec clés primaires et étrangères.

MPD (physique) : la **vraie base MySQL** avec types, longueurs et contraintes.

Q11. Quel est votre domaine d'étude ?

« L'informatisation de la gestion des clients de l'entreprise Wiky Holding. »

Q12. Qu'est-ce qu'un dictionnaire de données ?

La **liste de toutes les informations élémentaires** à stocker, chacune avec son code, sa signification, son type, sa longueur et sa nature. Le mien contient **26 propriétés distinctes**, sans doublon et sans donnée calculée.

Q13. Pourquoi ne mettez-vous aucune donnée calculée ?

Une donnée calculable (un total, une TVA) peut se **retrouver par calcul** : la stocker créerait un risque d'incohérence si une valeur change. Elle n'a donc pas sa place dans le dictionnaire ni dans le MCD.

Q14. Qu'est-ce qu'une SAT et quelle différence avec le GDF ?

Le **GDF** montre « si je connais A, je connais B ». La **SAT** (Structure d'Accès Théorique) est le GDF **sans les raccourcis inutiles** (les dépendances transitives).

Chez moi, **numMessage** et **numDevis** mènent chacun à numProspect, codeService et matAdmin. Il n'y a aucun raccourci à supprimer : **le GDF est égal à la SAT**.

Q15. Pourquoi n'y a-t-il aucun lien entre MESSAGE et DEMANDE_DEVIS ?

Parce que **aucune règle de gestion** du métier ne les relie : un message de contact et une demande de devis sont deux dossiers indépendants. **Ce qui n'a pas été dit dans les règles de gestion ne s'invente pas.**

Q16. Expliquez les cardinalités (0,n), (1,1) et (0,1).

Premier chiffre = **minimum** (0 facultatif, 1 obligatoire). Second = **maximum** (1 un seul, n plusieurs).

Exemple : « un prospect émet **zéro ou plusieurs** messages » (0,n) ; « un message est émis par **un et un seul** prospect » (1,1).

Q17. Pourquoi un message concerne au plus un service (0,1) alors qu'un devis porte sur un service (1,1) ?

Un visiteur peut écrire un message **général**, sans service précis. Mais une demande de **devis** n'a de sens que pour **un service précis** à chiffrer, donc le lien est obligatoire.

Q18. Qu'est-ce qu'une clé primaire et une clé étrangère ?

Clé primaire : identifiant unique d'une ligne (numProspect).

Clé étrangère : champ qui pointe vers la clé primaire d'une autre table (numProspect dans MESSAGE). Elle relie les tables et garantit l'intégrité des données.

Q19. Pourquoi vos clés étrangères acceptent-elles NULL dans MySQL alors que certaines cardinalités sont (1,1) ?

Au niveau **conceptuel**, le lien est obligatoire. Au niveau **physique**, j'ai choisi **ON DELETE SET NULL** : si un prospect ou un service est supprimé depuis le Back-Office, le message ou le devis **n'est pas perdu**, son lien devient simplement vide.

C'est un **choix d'implémentation** pour conserver l'historique.

Astuce : Question piège classique : réponds calmement, c'est un choix assumé et expliqué.

Q20. Pourquoi les tables messages et devis_requests contiennent-elles des colonnes (name, email, phone…) qui ne sont pas dans votre MLD-R ?

Ce sont des champs **du formulaire tel que le visiteur l'a rempli**, conservés tels quels pour garder la trace de ce qui a été envoyé, même si la fiche prospect change ou est supprimée. Le MLD-R est le **modèle de référence** ; le MPD a été **enrichi** pour les besoins du site.

Astuce : Si le jury insiste, reconnais : « C'est une petite redondance volontaire, à normaliser dans une prochaine version. »

Q21. Pourquoi des identifiants comme matAdmin, numProspect, codeService ?

MERISE impose des **familles de préfixes** : **mat** pour le personnel, **num** pour les dossiers et entités externes, **code** pour les catalogues, **id**, et **immat** pour les engins. Cela évite les homonymes et dit tout de suite de quoi parle l'identifiant.

Q22. Qu'est-ce que la 3FN ?

La **troisième forme normale** : chaque information dépend **directement et uniquement** de l'identifiant de sa table. En pratique : **une information, un seul endroit**, donc pas de doublons à corriger.

C. Architecture et programmation

Q23. Qu'est-ce que le MVC ?

Un patron d'architecture qui sépare le code en trois : **Modèle** (accès aux données via PDO/MySQL), **Vue** (affichage HTML), **Contrôleur** (reçoit la demande, vérifie la sécurité, appelle le modèle, envoie le résultat à la vue).

Image : le **serveur**, le **cuisinier** et l'**assiette** d'un restaurant.

Q24. Quel est le rôle du fichier index.php ?

C'est le **contrôleur frontal et routeur** : toutes les demandes passent par lui. Il lit le paramètre ?page= et appelle le bon contrôleur (HomeController, ContactController…). Si l'adresse n'existe pas, il affiche la page 404.

Q25. Qu'est-ce que PDO et pourquoi l'utiliser ?

PDO est la **couche standard de PHP pour parler aux bases de données**. Je l'utilise pour ses **requêtes préparées**, qui protègent contre l'injection SQL.

Q26. Pourquoi une classe Database et qu'est-ce que le Singleton ?

Tous les paramètres de connexion (hôte, nom de base, utilisateur, mot de passe, encodage) sont **écrits à un seul endroit** : app/Config/Database.php. Pour passer du local à l'hébergement, je ne modifie qu'un fichier.

Le **Singleton** garantit **une seule connexion** partagée par tous les modèles.

Q27. Pourquoi MySQL avec InnoDB ?

MySQL est un **standard du web**, gratuit, disponible sur WampServer comme sur InfinityFree. **InnoDB** respecte les propriétés **ACID** (fiabilité des opérations) et gère les **clés étrangères**.

Q28. Pourquoi PHP 8.2 ou plus, et que veut dire « typage strict » ?

Les versions récentes de PHP corrigent des failles des anciennes versions. Le **typage strict** oblige à déclarer le type de chaque donnée (texte, nombre, tableau) : cela évite des comportements imprévus.

Q29. Pourquoi ne pas avoir utilisé un framework ?

Pour garder un site **léger et rapide** (objectif : réponse sous 500 ms), **comprendre et maîtriser** chaque brique, et ne pas dépendre d'outils comme Composer ou SSH, souvent absents des hébergements gratuits.

Q30. Comment fonctionne le simulateur de devis ?

Le visiteur choisit le **service**, l'**envergure** (starter, standard, premium) et le **niveau d'urgence**. Du **JavaScript** recalcule le tarif indicatif en FCFA **sans recharger la page**. À l'enregistrement, la méthode **estimatePrice()** du modèle DevisModel refait le calcul côté serveur pour garantir la cohérence.

Q31. Qu'est-ce que le CRUD et où est-il dans votre projet ?

Create, Read, Update, Delete. Dans le Back-Office : les formulaires **créent** les messages et devis, le tableau de bord les **affiche**, on **modifie** leur statut, on les **supprime** (un par un ou en groupe).

Q32. Comment fonctionne la recherche instantanée (Ctrl + K) ?

Un raccourci clavier ouvre une barre de recherche. À chaque lettre tapée, le **JavaScript filtre en direct** les pages et prestations du site, sans recharger.

Q33. Comment le site s'adapte-t-il aux téléphones ?

Grâce au **responsive design** : grilles CSS modernes, unités relatives (rem, clamp) et un menu latéral rétractable sur smartphone (figure 28).

Q34. Qu'est-ce qu'une session ?

Une **mémoire côté serveur** qui retient qu'un utilisateur est connecté. Elle sert à l'administrateur, et à stocker le jeton CSRF.

D. Sécurité

Q35. Quelles protections avez-vous mises en place ?

Cinq principales : **requêtes préparées PDO** (injection SQL), **jetons CSRF**, **échappement** des affichages (XSS), **BCRYPT** pour les mots de passe, et **.htaccess** pour le serveur Apache. S'y ajoutent le cloisonnement des sessions et des messages d'erreur uniformes.

Q36. Expliquez l'injection SQL et votre protection.

Un pirate écrit une commande SQL dans un champ de formulaire pour tromper la base. Avec les **requêtes préparées**, la structure de la requête et les valeurs sont **envoyées séparément** : la valeur n'est jamais

exécutée comme une commande. J'ai aussi **désactivé l'émulation** pour que MySQL traite lui-même la requête préparée.

Q37. Expliquez le CSRF et votre protection.

C'est une attaque qui fait envoyer un formulaire à l'insu d'une personne connectée. Chaque formulaire contient un **jeton aléatoire de 32 octets** (random_bytes), stocké en session et comparé à la soumission avec **hash_equals**. Sans bon jeton, la requête est rejetée.

Q38. Expliquez le XSS et votre protection.

C'est l'injection d'un petit programme dans un texte affiché à d'autres personnes. J'**échappe** tout texte à l'affichage avec la fonction e() basée sur **htmlspecialchars** : le texte reste du texte.

Q39. Quelle différence entre hachage et chiffrement ? Peut-on retrouver un mot de passe ?

Un **chiffrement** peut se déchiffrer avec une clé. Un **hachage** comme **BCRYPT** est **à sens unique** : on ne peut pas retrouver le mot de passe, seulement vérifier qu'un mot de passe saisi donne la même empreinte.

Q40. Avez-vous une protection contre le brute force ?

Honnêtement : mon site **ne donne aucun indice** (message unique « Identifiants incorrects »), ce qui rend l'attaque plus difficile, et BCRYPT la ralentit. En revanche, **je n'ai pas encore de blocage après plusieurs échecs**.

C'est une **amélioration possible** : limiter les tentatives par adresse IP ou par compte.

Astuce : Dire clairement une limite est mieux perçu qu'un faux « c'est parfaitement sécurisé ».

Q41. Votre mot de passe administrateur est « admin123 » dans le rapport : est-ce sûr ?

C'est un **compte de test pour la recette** en local. En production, l'administrateur **change son identifiant et son mot de passe** depuis la fenêtre dédiée du tableau de bord, avec confirmation de l'ancien mot de passe et nouveau hachage BCRYPT immédiat.

Astuce : Conseil pratique : avant la soutenance, change ce mot de passe et ne l'affiche pas à l'écran.

Q42. Que fait le .htaccess ?

C'est le fichier de configuration d'Apache. Chez moi : désactive le **listing des dossiers**, masque la signature du serveur, **bloque l'accès direct** aux fichiers sensibles (database.sql…), ajoute des **en-têtes de sécurité** et gère la **réécriture d'URL**.

E. Tests, déploiement et bilan

Q43. Comment avez-vous testé le site ?

J'ai fait une **matrice de 14 tests** : chargement des pages, routage, connexion MySQL, formulaires de contact et de devis, connexion admin réussie et échouée, changement de statut, suppression groupée, changement de mot de passe, purge, protection CSRF, affichage PC/mobile, page 404. Tous sont **conformes**.

Q44. Comment déploie-t-on le site ?

En local : installer WampServer, copier le dossier dans www, créer la base wiky_db dans phpMyAdmin, importer database.sql, ouvrir http://localhost/achelyrapport/.

En ligne : compte InfinityFree, créer la base MySQL, importer database.sql, renseigner les 4 constantes dans Database.php, transférer les fichiers par FileZilla.

Q45. Pourquoi InfinityFree et quelles sont ses limites ?

C'est un hébergement **gratuit** avec MySQL, PHP et certificat SSL : idéal pour un projet de stage. Limites : **pas de SSH ni de Composer**, ressources limitées, donc pas adapté à un très fort trafic. Une agence en production choisirait plus tard un hébergement payant.

Q46. Le coût est de 0 FCFA : vraiment ?

Le coût **matériel et logiciel** est nul car j'ai utilisé des outils libres et gratuits. Le vrai investissement est **mon temps** : analyse, modélisation, programmation, tests et documentation.

Q47. Quelles difficultés avez-vous rencontrées ?

La principale : **repartir de zéro** en code PHP après la perte du site WordPress, tout en respectant les limites des hébergements gratuits et la rigueur attendue en BTS.

Q48. Qu'est-ce que ce stage vous a apporté ?

La pratique de **MERISE** du dictionnaire au MLD-R, l'**architecture MVC**, la **POO en PHP** et PDO, la **sécurité web**, le **responsive design**, et la découverte du **fonctionnement d'une agence** de communication.

Q49. Quelles améliorations envisagez-vous ?

Notifications **email et SMS** à chaque demande, **factures proforma PDF** depuis le tableau de bord, **catalogue de réalisations** géré depuis le Back-Office, **version bilingue** français/anglais. À cela, je peux ajouter la **limitation des tentatives de connexion**.

Q50. Que se passe-t-il si le serveur MySQL est arrêté ?

La classe Database **intercepte l'erreur** et affiche un écran clair en français qui guide pour vérifier que les services WampServer sont démarrés, au lieu d'une page d'erreur illisible.

Q51. Comment le site peut-il être mieux trouvé sur Google ?

Les **URL sont lisibles** grâce à la réécriture, les pages ont des **titres et contenus structurés** en HTML5 sémantique, et le site est **rapide**. Ce sont les bases du référencement ; on pourrait ajouter plan de site et balises dédiées.

Q52. En quoi votre site est-il différent d'un site WordPress ?

Il est **sur mesure** : pas de modules inutiles, plus **léger**, avec une **base conçue par MERISE** et une sécurité maîtrisée. WordPress est plus rapide à mettre en place mais plus généraliste et plus exposé s'il n'est pas mis à jour.

Q53. Si le site recevait beaucoup plus de visiteurs, que faudrait-il faire ?

Passer à un **hébergement plus puissant**, ajouter de la **mise en cache** et optimiser les requêtes. L'architecture MVC et les **index MySQL** facilitent cette évolution.

Partie 6 — La démonstration, pas à pas

La démo dure **3 à 4 minutes**. Elle prouve que tout fonctionne. L'objectif n'est pas de tout montrer, mais de **raconter un parcours** : un visiteur arrive, envoie une demande, et l'administrateur la traite.

6.1 Avant la soutenance (à faire la veille et 30 minutes avant)

1. **Démarre WampServer** : l'icône dans la barre des tâches doit être **verte**.
2. Vérifie que **http://localhost/achelyrapport/** s'ouvre et que la base **wiky_db** est bien importée.
3. **Change le mot de passe admin** et note-le ailleurs que sur l'écran.
4. Ouvre **à l'avance** les onglets : accueil, page Devis, page de connexion admin.
5. Branche le chargeur, désactive les notifications, mets le son en silencieux.
6. **Prépare un plan B** : les captures d'écran du rapport (figures 8 à 28) sur une diapositive de secours si la démo plante.

Important

- **Ne clique jamais sur « Purger TOUT »** devant le jury : ce bouton vide les tables.
- Ne montre le bouton « Injection de 10 données de test » qu'**avant** la soutenance, pour que le tableau de bord soit déjà rempli.

6.2 Le scénario de démonstration

#	Tu fais	Tu dis
1	Page **d'accueil**	« Voici la vitrine : bannière, boutons d'appel à l'action, résumé des quatre pôles. »
2	Page **Services** puis **Réalisations** : clique sur un filtre	« Les services et trois packs. Ici le **filtre** est instantané, sans recharger la page : c'est du JavaScript. »
3	Appuie sur **Ctrl + K**, tape quelques lettres	« Une **recherche instantanée** sur les pages et prestations. »
4	Change le **thème** (clair / sombre / bleu marine)	« Le choix est mémorisé dans le navigateur (localStorage). »
5	Réduis la fenêtre (ou montre la figure 28 mobile)	« Le site est **responsive** : il s'adapte au téléphone. »
6	Page **Contact** : remplis et envoie un message	« Le message est enregistré en base **de façon sécurisée** : jeton CSRF, requête préparée. »
7	Page **Devis** : change le service et l'urgence, regarde le prix bouger, envoie	« Le **simulateur** calcule en direct en FCFA. La demande est enregistrée dans la table devis_requests. »
8	Ouvre la page **connexion admin**, connecte-toi	« L'accès est protégé : mot de passe haché en BCRYPT, aucun indice affiché, message d'erreur unique. »
9	Tableau de bord : montre les **KPI**	« Messages non lus, devis en attente, état de MySQL, total des enregistrements. »
10	Ouvre la **demande que tu viens d'envoyer**, change son **statut**	« Je consulte dans une fenêtre modale et je passe de 'En attente' à 'Validé' : c'est le **CRUD**. »

#	Tu fais	Tu dis
11	(Option) Montre phpMyAdmin et le concepteur (figure 3)	« Voici les **5 tables** et leurs liens : c'est mon MCD devenu réalité. »

Phrase de transition vers la conclusion

« Vous avez vu le parcours complet : **du visiteur qui écrit, jusqu'à l'administrateur qui traite la demande**. Tout repose sur une base de données modélisée avec MERISE et sécurisée. »

Partie 7 — Conseils du jour J et fiche mémo

7.1 La veille et le matin

- **Dors.** Un cerveau reposé répond mieux qu'un cerveau qui a tout relu à 3 heures du matin.
- Relis **uniquement la fiche mémo** ci-dessous et le pitch de 2 minutes.
- Fais **une répétition complète à voix haute**, chronomètre en main.
- Imprime ton rapport, avec des **marque-pages** sur : SAT (p. 19), MCD (p. 20), tables MySQL (chap. IV), sécurité (chap. VII), tests (chap. VIII).
- Tenue correcte, arrive **30 minutes en avance**, prends de l'eau.

7.2 Pendant la soutenance

- **Écoute la question jusqu'au bout.** Reformule si besoin : « Si je comprends bien, vous me demandez… ».
- **Réponds en 3 temps :** la réponse en une phrase → l'explication → un exemple de ton projet.
- **Utilise tes images** : classeur (base de données), restaurant (MVC), plans de maison (MERISE), badge (session), cadenas (HTTPS).
- **Sois honnête sur les limites** (pas de blocage anti-brute force, hébergement gratuit) : cela montre ta lucidité.
- **Ne dis jamais** « je ne sais rien » ni « c'est mon encadrant qui l'a fait ». Dis ce que **tu** comprends et ce que **tu** as fait.
- Si tu te trompes, corrige-toi calmement : « Pardon, je reformule… ».

7.3 Le vocabulaire à utiliser avec aisance

Emploie ces mots, ils donnent une image de maîtrise : **modélisation, intégrité référentielle, normalisation (3FN), requête préparée, séparation des responsabilités, maintenabilité, évolutivité, jeton CSRF, échappement, hachage, responsive, recette (phase de test), déploiement, back-office**.

7.4 Fiche mémo (à relire la veille)

Le projet en 10 lignes

- **Qui :** Kouame Affoue Cyntiche Achley Delmas, BTS IDA, ES2I Plateau.
- **Où :** Digital Agency Wiky (Wiky Holding), Abidjan, Cocody Riviera Faya. Agence de communication 360°.
- **Quoi :** site web avec 7 pages, formulaires de contact et de devis, simulateur de prix en FCFA, Back-Office sécurisé.
- **Comment :** PHP 8.2+ orienté objet, MVC, MySQL InnoDB, PDO, JavaScript, HTML/CSS responsive.
- **Modélisation :** MERISE : 26 propriétés, 6 règles de gestion, SAT (GDF = SAT), MCD 5 entités / 6 associations, MLD-R en 3FN.
- **Tables :** services, prospects, admins, messages, devis_requests.
- **Sécurité :** requêtes préparées PDO, CSRF (jeton 32 octets), XSS (échappement), BCRYPT, .htaccess, session_regenerate_id.
- **Tests :** 14 scénarios conformes. **Outils :** WampServer 3.4.0, phpMyAdmin, InfinityFree, FileZilla.
- **Coût :** 0 FCFA. **Perspectives :** email/SMS, factures PDF, catalogue dynamique, bilingue, anti-brute force.

Les 6 images à retenir

- **Base de données** = un classeur à 5 tiroirs.
- **MERISE** = les plans de la maison avant de construire.
- **MVC** = le serveur, le cuisinier et l'assiette.
- **Requête préparée** = le modèle de la question d'abord, les valeurs ensuite, sans mélange.
- **Jeton CSRF** = un tampon secret sur chaque formulaire.
- **BCRYPT** = une empreinte irréversible : on peut comparer, pas relire.

Les 3 cardinalités et les 3 clés

- **(0,n)** peut en avoir plusieurs ou aucun · **(1,1)** un et un seul · **(0,1)** au plus un.
- **mat** = personnel · **num** = dossier / entité externe · **code** = catalogue.
- **SAT** = GDF sans raccourcis transitifs. Chez toi : GDF = SAT.

Tu connais ton projet, tu l'as vécu, tu l'as testé. Respire, souris, explique simplement. Bonne soutenance !